

CO1020 Computer Systems Programming

Group 48 Project Report

3D Software Renderer in C (LibTiny3D)

E/22/179, E/22/205

University of Peradeniya, Department of Computer Engineering

Date: July 7, 2025

Contents

1	Task 1: Canvas and Line Drawing	3
2	Task 2: 3D Math Library	5
3	Task 3: Rendering and Lighting	7
4	Task 4: Animation and Demo	9
5	Design Decisions	10
6	AI Tool Usage	10
7	Individual Contributions	10

1. Task 1: Canvas and Line Drawing

Overview

This task involved creating a digital drawing surface (canvas) and implementing smooth line rendering. Unlike pixelated integer-only drawing, our canvas supports floating-point coordinates like $(3.5, 4.2)$ and renders soft lines using interpolation.

3.1.1 Implementation Details

`canvas_t` Structure

Implemented in `canvas.h`, the canvas structure stores:

- Width and height of the image
- A 2D float array where each value is a pixel brightness between 0.0 (black) and 1.0 (white)

This float-based representation enables subpixel rendering.

`set_pixel_f(canvas, x, y, intensity)`

Implemented in `canvas.c`, this function:

- Accepts floating-point coordinates (x, y)
- Spreads the brightness `intensity` across the 4 neighboring integer pixels using **bilinear interpolation**

Bilinear Interpolation: Let (i, j) be the floor of (x, y) . The brightness B is distributed to 4 neighbors:

$$w_{00} = (1 - dx)(1 - dy), \quad w_{10} = dx(1 - dy), \quad w_{01} = (1 - dx)dy, \quad w_{11} = dx \cdot dy$$

where:

$$dx = x - i, \quad dy = y - j$$

Then the 4 surrounding pixels receive:

$$\begin{aligned} B_{i,j} &= w_{00} \cdot \text{intensity} \\ B_{i+1,j} &= w_{10} \cdot \text{intensity} \\ B_{i,j+1} &= w_{01} \cdot \text{intensity} \\ B_{i+1,j+1} &= w_{11} \cdot \text{intensity} \end{aligned}$$

`draw_line_f(canvas, x0, y0, x1, y1, thickness)`

Also in `canvas.c`, this function uses the **Digital Differential Analyzer (DDA)** algorithm to draw lines between two points with subpixel accuracy.

DDA Algorithm: To draw a line from (x_0, y_0) to (x_1, y_1) :

$$\begin{aligned}\Delta x &= x_1 - x_0, & \Delta y &= y_1 - y_0 \\ \text{Steps} &= \max(|\Delta x|, |\Delta y|) \\ \text{Increment: } dx &= \frac{\Delta x}{\text{steps}}, & dy &= \frac{\Delta y}{\text{steps}}\end{aligned}$$

We loop over steps and draw points at:

$$(x_k, y_k) = (x_0 + k \cdot dx, y_0 + k \cdot dy), \quad k = 0, 1, \dots, \text{steps}$$

Each point is drawn using `set_pixel_f` to blend intensities smoothly.

Line Thickness

Thickness is simulated by drawing multiple parallel lines perpendicular to the main direction, using offset sampling.

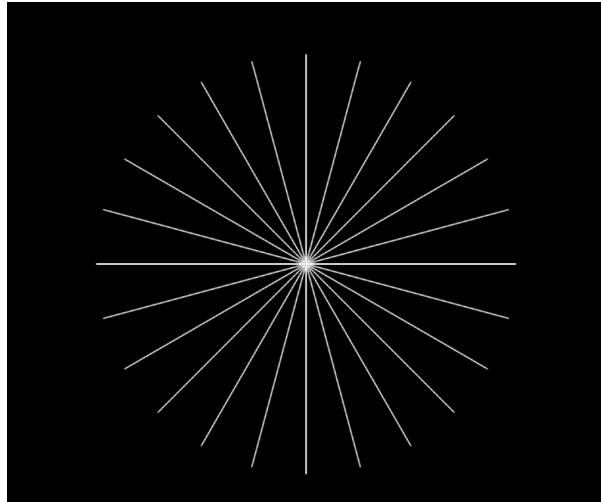


Figure 1: clock faces

2. Task 2: 3D Math Library

Overview

In this task, we developed the foundational mathematics for 3D rendering, including vector and matrix types, operations, and transformations.

vec3_t Structure: Cartesian and Spherical Coordinates

The `vec3_t` structure holds a 3D vector in both:

- Cartesian form: (x, y, z)
- Spherical form: (r, θ, ϕ)

Whenever one representation is updated, the other is synchronized using conversion formulas.

Cartesian $\rightarrow$ Spherical

$$r = \sqrt{x^2 + y^2 + z^2}, \quad \theta = \arccos\left(\frac{z}{r}\right), \quad \phi = \arctan 2(y, x)$$

Spherical $\rightarrow$ Cartesian

$$x = r \sin \theta \cos \phi, \quad y = r \sin \theta \sin \phi, \quad z = r \cos \theta$$

vec3_from_spherical

This function constructs a `vec3_t` from spherical coordinates using:

$$x = r \sin \theta \cos \phi, \quad y = r \sin \theta \sin \phi, \quad z = r \cos \theta$$

It is especially useful when animating orbiting objects or camera positioning.

vec3_normalize_fast

This function normalizes a vector to unit length using an approximate inverse square root:

$$\vec{v}_{\text{normalized}} = \frac{\vec{v}}{\sqrt{x^2 + y^2 + z^2}} \approx \vec{v} \cdot \text{Q_rsqrt}(x^2 + y^2 + z^2)$$

The fast inverse square root trick (popularized by Quake III) provides high performance with reasonable accuracy.

vec3_slerp

Spherical linear interpolation (slerp) is used to interpolate smoothly between two directions:

$$\text{slerp}(a, b, t) = \frac{\sin((1-t)\omega)}{\sin(\omega)}a + \frac{\sin(t\omega)}{\sin(\omega)}b$$

Where $\omega = \cos^{-1}(a \cdot b)$ is the angle between the vectors. This is useful for turning, morphing, and animation.

mat4_t Structure and Matrix Operations

We represent 4x4 matrices in column-major layout, following OpenGL-style transformations. Each transformation matrix is defined and used for combining model, view, and projection stages.

mat4_identity

Generates a 4×4 identity matrix:

$$I = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

mat4_translate(tx, ty, tz)

Builds a translation matrix:

$$T = \begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

mat4_scale(sx, sy, sz)

Builds a scaling matrix:

$$S = \begin{bmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

mat4_rotate_xyz(rx, ry, rz)

Creates combined rotation about x , y , and z axes. Each rotation matrix:

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(rx) & -\sin(rx) & 0 \\ 0 & \sin(rx) & \cos(rx) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_y, R_z \text{ similar}$$

Final rotation matrix is:

$$R = R_z \cdot R_y \cdot R_x$$

mat4_frustum_asymmetric

This builds a perspective projection matrix from frustum parameters:

$$P = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Where:

- l, r = left/right
- b, t = bottom/top
- n, f = near/far

This allows customized asymmetric perspective projections suitable for special views like isometric or stereoscopic rendering.

3. Task 3: Rendering and Lighting

Overview

Developed the core wireframe rendering pipeline with lighting. Used projection, model/view transformations, and Lambertian lighting.

Projection

Perspective projection (simple):

$$x' = \frac{x}{-z}, \quad y' = \frac{y}{-z}$$

Then scaled to screen coordinates:

$$x_{screen} = \left(\frac{x' + 1}{2} \right) \cdot width$$

Lambertian Lighting

Each edge is evaluated based on:

$$I = \max(0, \vec{L} \cdot \vec{D})$$

where $\vec{L}$ is the light direction and $\vec{D}$ is edge direction.

Later modified to:

$$I = \frac{K}{1 + d^2}$$

Where d is distance from midpoint of the edge to the light source for realism.

Render Pipeline

- Project each vertex - Draw edges between visible vertex pairs - Apply brightness based on light direction and proximity

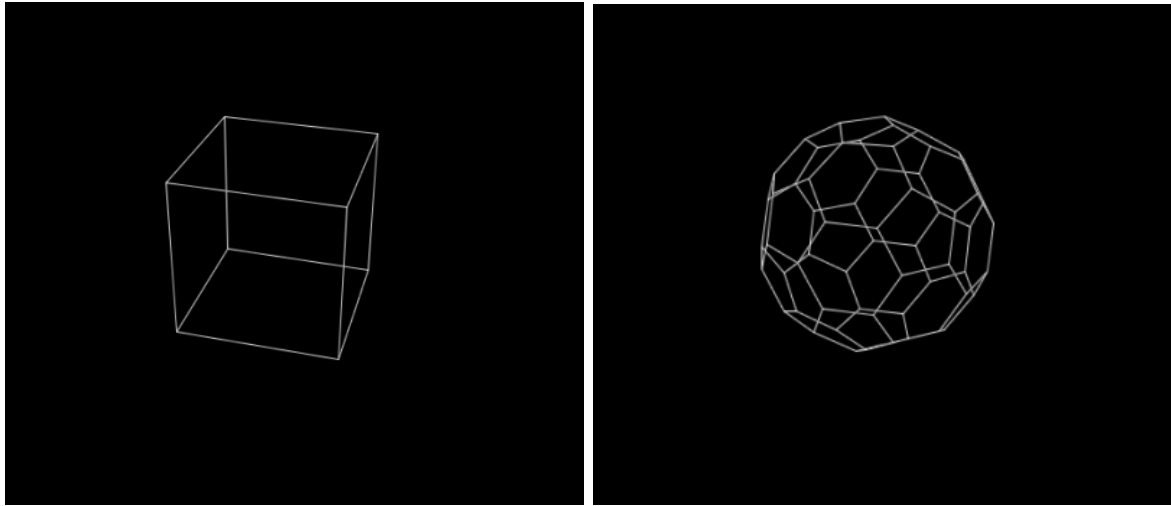


Figure 2: Rendered Soccer Ball and Cube

4. Task 4: Animation and Demo

Overview

This task includes Bézier-based animation and a full rotating cube and soccer ball demo.

Soccer Ball Bouncing Logic

Used:

$$y(t) = |\sin(\omega t)| \cdot A$$

Where $A = 2.0$, $\omega = 2\pi/T$

Orbit Movement

$$x(t) = R \cos(\theta), \quad z(t) = R \sin(\theta)$$

Cube Rotation

The cube rotates around the X-axis:

```
mat4_rotate_x(angle)
```

Demo

- Soccer ball bounces and orbits - Cube rotates at center - Fixed left-direction light

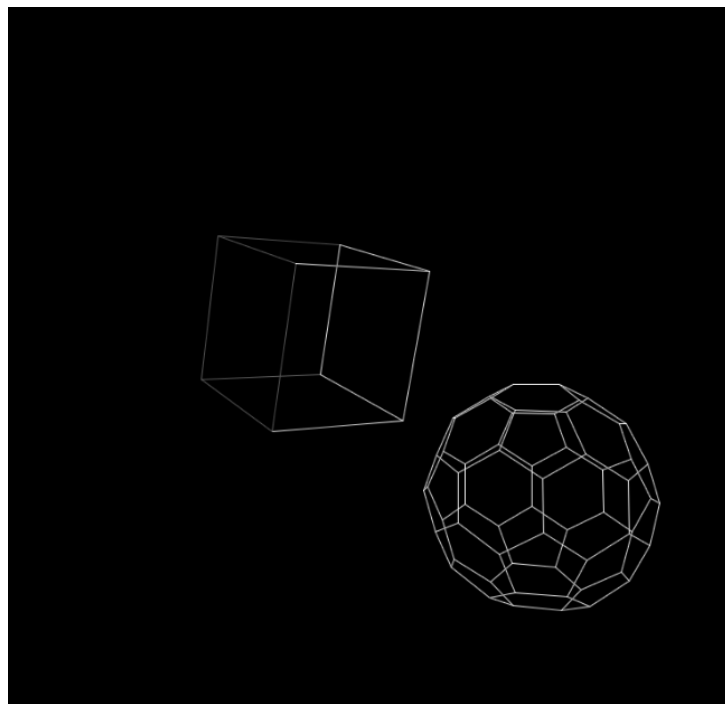


Figure 3: Soccer Ball and Cube in Final Animation

5. Design Decisions

- Used PGM for portability and simplicity
- Wireframe rendering allows full transparency for debugging geometry
- Lambert lighting is simple yet effective for edge shading
- Structured code into ‘canvas’, ‘math3d’, ‘renderer’, ‘lighting’, and ‘animation’ modules

6. AI Tool Usage

ChatGPT and GitHub Copilot were used for:

- Refining mathematical equations
- Suggesting improvements to vector/matrix functions
- Explaining and validating lighting techniques
- Structuring the Makefile and LaTeX report

7. Individual Contributions

E/22/179:

- Implemented math3d, canvas drawing, and lighting system
- Refined the cube and soccer ball animation code

E/22/205:

- Designed and rendered the soccer ball geometry
- Created ‘visualsoccerball.c’ and presentation
- Developed report and README documentation

References

- Computer Graphics: Principles and Practice, Foley et al.
- OpenAI ChatGPT for code guidance